Enhancing the Precision of a User's Hand When Sliding on Objects in VR

ANNA KOZŁOWSKA, MIKOŁAJ WÓJCIK, MACIEJ SPYCHAŁA, and JOANNA PORTER-SOBIERAJ, Warsaw University of Technology, Poland
JAKUB CIECIERSKI, Virtual Simulations, Poland

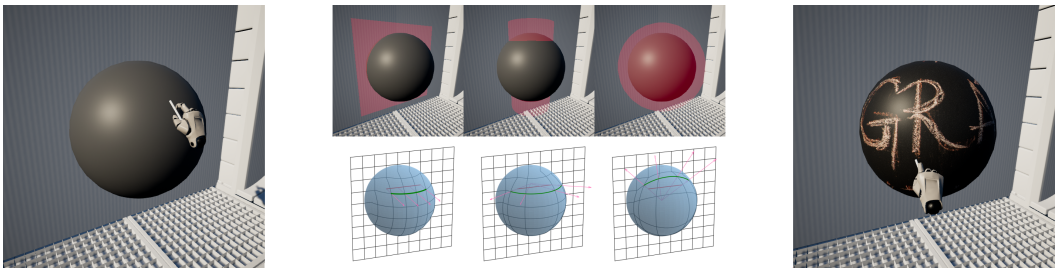


Fig. 1. Human-object interface based on auxiliary projection shapes. Different hand behavior is visible when interacting with the surface of the virtual sphere using a projection plane, a projection cylinder, and a projection sphere.

Interaction in virtual reality (VR) remains a major research challenge, due to the difficulty of accurately reproducing a user's position, the differences that exist between the virtual and real worlds, and the need to ensure a faster response than in traditional computer systems. Our work focuses on software-based approaches that support the user's hand running over VR objects. Existing constraint-based methods implemented in game engines result in visible glitches which negatively affect the user's immersion. To mitigate this, we have developed an easy-to-implement human-object interface based on auxiliary projection shapes. Our method, like other software-centered solutions, does not require any additional advanced haptic hardware; it relies solely on a standard VR headset (HMD) with controllers. The evaluation performed confirmed that our approach allows smooth motion to be attained in a 3D scene without adding additional latency to the VR system. Furthermore, for different projection shapes, different hand behavior on the same 3D object is obtained. As a result, scene designers are offered a simple visual mechanism to manipulate hand motion on 3D objects' surfaces, enhancing its precision while sliding to create more desirable hand trajectories.

CCS Concepts: • **Human-centered computing → Interaction techniques**; *User studies*; • **Computing methodologies → Virtual reality**.

Additional Key Words and Phrases: Virtual Reality, Graphical Human-Computer Interfaces, Graphical Interaction, Constraints, Shape Analysis

ACM Reference Format:
Anna Kozłowska, Mikołaj Wójcik, Maciej Spychała, Joanna Porter-Sobieraj, and Jakub Ciecierski. 2025. Enhancing the Precision of a User's Hand When Sliding on Objects in VR. *Proc. ACM Comput. Graph. Interact. Tech.* 8, 1, Article 2 (May 2025), 18 pages. https://doi.org/10.1145/3728307

Authors' Contact Information: Anna Kozłowska, anna.kozlowska@pw.edu.pl; Mikołaj Wójcik, mikolaj.wojcik@pw.edu.pl; Maciej Spychała, maciej.spychala@pw.edu.pl; Joanna Porter-Sobieraj, joanna.porter@pw.edu.pl, Warsaw University of Technology, Warsaw, Poland; Jakub Ciecierski, j.ciecierski@vea-sim.com, Virtual Simulations, Warsaw, Poland.

1 Introduction

Virtual reality (VR) applications allow users to interact with virtual environments in ways that are impossible to recreate in classic desktop or mobile applications, and they achieve this through the use of specialized graphical human-computer interfaces such as headsets supplemented with dedicated hand-tracking controllers. Various 3D object selection and UI interaction techniques are explored by Argelaguet and Andujar [2013].

VR allows for numerous combinations of selection and interaction techniques, e.g.: (1) a combination of raycasting as a selection mechanism with "pulling" causing objects to be grabbed by the users, (2) a combination of proximity based selection (the selected object is the one closest to the hand) with socket like interactions - predefined positions in which object slots into the hand of the user, (3) a physical selection with physical grabbing which aims to replicate grabbing as it occurs in the real world.

Additionally, we can distinguish the following interactions: (1) *grabbing* and manipulating objects, (2) *pushing* objects around via contact, and (3) *running a hand over* objects to feel their texture. *Grabbing* focuses on selecting and picking up an object, holding it in one or two hands, rotating, moving it from one place to another, and any other action that requires picking it up first. By *pushing* an object via contact, we mean actions in which interaction is applied via a collision with the user's body (e.g. a hand or a finger), resulting in the object's movement (e.g. pressing a clickable button, pushing crates, or opening doors). *Running a hand over* objects is an interaction in which the user touches the object but does not change its position or shape. A good example of this interaction is sliding a hand over a wall or other static object. In such cases, the object is not modified regardless of the applied force. In this paper, such interactions will be referred to as *sliding*.

One of the main problems that requires consideration during the creation of a VR application is the difference between a *virtual* world and the *real* one. The objects visible in the virtual world do not correlate exactly with these in the real world, and vice versa. Users can collide with objects in the real world while being in VR, where the space is unoccupied, and can pass through objects in VR because they do not exist in the real world. These problems have to be considered when creating VR environments because they can adversely affect users' immersion.

Therefore, nowadays, when it comes to VR applications, the main challenges are (1) how to provide a workable solution to the problem of collisions between a user's avatar and a virtual 3D environment, and (2) how to reflect these collisions in the real world. Any mismatches between the real and virtual world lead to ruptures in the immersion and spoil the user's experience. There already exist mechanical solutions in the form of haptic devices which allow developers to restructure user's movement restrictions. Unfortunately, they are still expensive, and are inconvenient to set up and use. For example, tactile and force feedback gloves can provide the user with an imitation of the sense of touch. However, usually, they cannot prevent the user from putting their hand inside a virtual object since they do not provide kinesthetic feedback [Caeiro-Rodríguez et al. 2021; Iacob et al. 2014]. Known solutions include gloves made by HaptX, Manus, BHaptics, and Diver-X, but they are beyond the budget of most users. And the few that are affordable are still in the prototype stage. Some users decide to use motion capture gloves, e.g., those produced by Rokoko, because while they do not provide haptic feedback, the sense of agency added by tracking hands is acceptable. When haptic feedback is provided, the user can still find ways to put their hand inside the object, e.g. by simply taking a step forward or moving their whole arm forward.

In older or out-of-the-box solutions, collisions between a virtual hand's trajectory and virtual objects in the 3D scene are either neglected or solved via a naive approach using generic collision solvers developed for non-VR applications. When the user's hand collides with the object, it is

expected to be stopped and remain on the object's surface. Ignoring any collisions between the avatar and the virtual scene thus causes an inauthentic user experience. With generic collision solvers [Epic Games, Inc. 2023; Unity Technologies 2023], a hand mesh is prevented from further movement when the first collision is detected, and later, its position is projected onto the nearest surface using these surface parameters. Both ignoring or solving collisions in a generic way can cause a non-authentic experience that ruins user immersion.

In this paper, we focus on the *sliding* interaction in virtual reality, which is closely related to haptic rendering [Srinivasan and Basdogan 1997]. The sense of touch is critical for proper spatial orientation, and people frequently use it subconsciously. Additionally, the sense of touch allows one to feel the texture of the object and its firmness. We aim to create methods that can be used by the consumer while only utilizing ordinary VR controllers and headsets. We propose a new method for correcting the user's hand trajectory in response to collisions with virtual objects in the scene. Our method allows the scene designer to control the way in which any corrections are performed. With this method, the experience of interacting with objects is improved so that the user's hand can smoothly slide along (run over) the outside of the colliding surface of the object instead of penetrating it. The precision of the user's hand movements, especially in the case of dynamic ones, considerably improves upon standard methods implemented in game engines.

2 Related work

The quality of the VR experience depends heavily on the hardware and software employed. The most important factors are a high and stable refresh rate of the generated image, and precise tracking of the user position. New technologies such as *foveated rendering* [Franke et al. 2021; Waisberg et al. 2024; Wang et al. 2023], *gaze analysis* [Hu 2020; Kar and Corcoran 2017] and *image reprojection/timewarp* [Franke et al. 2021; van Waveren 2016] help bring a high-quality VR experience to the masses. Unfortunately, even the most technologically advanced headsets and controllers cannot provide a true feeling of physical interaction with a virtual world.

The first techniques for bringing more realism to virtual reality date back to the end of the 20th century [Buck and Schömer 1998; Turner and Gobbetti 1998]. One of the main concerns in those considerations, which still needs to be solved, is how to make interactions in the virtual environment as similar as possible to those in the real world. One issue is that unnatural hand or finger movements increase the user's fatigue [Kim and Park 2014]. We can observe two main approaches to this problem: hardware and software.

Hardware solutions, such as force feedback haptic systems [Bouzit et al. 2002; Murayama et al. 2004] are widely used to improve the user's perception of interactions. Such hardware allows the user to perform real-world-like tasks in VR in an intuitive manner in the same way in the real world and with realistic stimuli. This allows for blocking the user's movement and makes use of the sense of touch - not used in ordinary VR hardware - to increase user's immersion. Force feedback haptic systems include haptic gloves, ultrasound haptic systems, robots and more specialized hardware imitating real-world situations. Such hardware is usually made on a case-by-case basis: haptic gloves for manipulating objects, treadmills for running, or a moving plane cockpit for simulating G-forces.

In the work [Rafferty et al. 2017], a system for treating phobias is presented. The user can confront his or her phobia by climbing a virtual wall, while a Baxter robot's end effector provides a physical proxy for virtual ledges. In Devine et al. [2017], another haptic VR system which uses HTC Vive and a Baxter robot is presented. The research focuses on using force feedback to simulate the weight of any object in virtual reality. In Hoppe et al. [2018], a system utilizing quadcopters as a levitating haptic feedback proxy is presented. It simulates objects and their position in the virtual

scene and increases the user's immersion. Such an approach locally modifies the real world so that constraints from VR are also physically present in the real world.

In recent years, many hardware solutions which use gloves as human-machine interfaces (HMIs) and focus on the human hand's sense of touch have been developed [Ozioko and Dahiya 2022; Pacchierotti et al. 2017; Pan et al. 2022; Wang et al. 2019]. They concentrate, for example, on creating cheap and accurate glove systems [Choi et al. 2016; Zhu et al. 2020], reducing feedback latency, something greatly needed in remote surgery solutions [Sim et al. 2021] and on providing information about the texture of an object [Tanaka et al. 2023]. Gloves are becoming especially popular as they allow users to perform tasks in a natural way, just like in the real world [Bartalucci et al. 2023; Qi et al. 2023]. Due to size and weight constraints, and required feedback force - constraints that have only recently became solvable - they are still at the prototype stage. The results confirm that haptic gloves can substantially increase the user's immersion. However, due to their high price [Caeiro-Rodríguez et al. 2021; Chen et al. 2020; Perret and Vander Poorten 2018], software-centered methods of providing realism in VR applications are increasingly popular, with the aim of deepening the user's immersion in an application. Moreover, hardware solutions can be easily circumvented. For example, a force feedback glove can prevent the users from pushing their fingers into the object, but the user still can move their hand so that it gets inside the object. Such problems may only be solved by creating specialized equipment or employing a software-based approach.

Some researchers try to mitigate the lack, or high price, of haptic feedback devices by simulating them. As described in [Pusch et al. 2009], fast hand displacement is used to exploit the human brain's capability of adjusting received signals to form a compelling experience. The authors present this technique as a way to simulate the feeling of moving a hand in a force field, such as a river or stream. This topic is explored in-depth in [Gibbs et al. 2022; Kang et al. 2021], in which the combination of visual and haptic or auditory cues are found to be able to replace real haptic feedback.

Tong et al. [2023] investigates relevant studies on hand-based haptic interaction for virtual reality. While our research distinguishes different interaction types such as grabbing, moving through contact, and sliding over, most software solutions focus only on creating realistic grabbing of objects for VR applications [Prachyabrued and Borst 2012; Wang et al. 2022]. Grabbing interaction can be simulated using many different techniques and with a different degree of fidelity. In the work [Argelaguet et al. 2016], the sense of embodiment achieved while interacting with different virtual hand representations is tested. In the experiment, users employed different hand representations. These included an abstract sphere and a realistic model of a hand. It was proven that an abstract object such as a sphere with only 3 degrees of freedom provided an excellent sense of agency.

In [Jacobs and Froehlich 2011; Verschaar et al. 2018], the authors present a model hand, based on soft body simulation of fingers, that was computationally efficient and precise enough to allow for realistic object manipulation. Kim and Park [2015] presents a particle-based model for 3D object manipulation and grasping. Holl et al. [2018], meanwhile, describes a simplified and fast approach - the algorithm detects the intersection between the model hand and virtual objects and uses the Coulomb friction model to compute the resulting forces.

Software solutions allow for creating advanced and life-like interactions in which the user employs their hands. What is more, they are significantly cheaper than hardware solutions and, as such, may be used in consumer-grade equipment. Unfortunately, in some applications of VR technology, such as telerobotics, precision of movement is critical, and users are forced to use specialized equipment to achieve it.

The key contribution of our work includes a novel algorithm that enables smooth hand sliding on scene objects while simultaneously mimicking the user's movements correctly and intuitively, using only a standard VR headset with controllers.

3 Sliding the hand over

3.1 Problem statement

In our work, we aim to create smooth hand movement over objects located in a virtual scene. We focus only on a software approach since we wish to provide a cheap and user-friendly solution that requires only basic VR kit.

The input x_r in our algorithm is the position of the hand in the real world, e.g. provided by the VR controller. First, the position x_r of the hand in the real world is mapped onto the hand x_v in the virtual world:

$$x_r \overset{T_{map}}{\longmapsto} x_v. \tag{1}$$

The transformation T_{map} is handled by VR hardware and a VR game engine.

Naturally, when users move around in the virtual 3D space, their hand movement envelope often intersects with virtual obstacles, especially when they try to interact with the environment close to objects' surfaces. VR applications usually handle such cases via one of two approaches:

(1) *Ignoring* the collision completely, allowing the user's hand to pass through the object's volume freely.
(2) *Changing* the position of the virtual hand to stop it on the surface and to prevent the hand from penetrating scene objects.

The first approach, although possessing the advantage that the virtual hand position approximates the physical hand position exactly, has a very detrimental effect on the user's immersion because they expect their hand to collide with the object. Furthermore, the user may accidentally start interacting with objects that should not be accessible, for example, by moving their hand through a virtual wall and interacting with objects on the other side.

For a more immersive experience of the application, the second approach is often chosen. In this case, VR interfaces need to adjust hand position x_v to an adjusted position x_a in order to ensure that the hand geometry remains on the object's surface:

$$x_v \overset{T_{adj}}{\longmapsto} x_a. \tag{2}$$

Finally, only the adjusted hand x_a is rendered on the VR scene. Implementation of the position adjustment transformation T_{adj} is usually done by the game engine collision solver [Epic Games, Inc. 2023; Unity Technologies 2023] applying the physics-based approach used to simulate rigid body interactions [Baraff 1993, 1996; Harada et al. 1995; Redon et al. 2002]. The two most commonly used techniques are: *collider-based* and *constraint-based*.

The idea of a *collider-based* method is based on checking whether any collision of the virtual hand with the objects in the virtual world occurred and - if so - getting the virtual hand x_v back on the nearest surface to avoid an intersection for every hand position independently. To project the virtual hand back on the object surface, the surface's normal vector is usually used.

However, the trajectory $x_a(t)$, obtained as a projection of the hand's trajectory $x_v(t)$ along the surface's normal vector, may be discontinuous, e.g. at the edge where the two surfaces of the object meet (Figure 2). As a result, the adjusted hand $x_a(t)$ not only appears to move at vastly different speeds despite the constant speed of the user's movements $x_v(t)$ but also rapidly jumps between two positions, A and B. Both of these positions are the projections of point $P \in x_v(t)$ for which a unique projection does not exist. This problem may be mitigated by smoothing polygonal meshes [Morgenbesser and Srinivasan 1996].

Another issue that is even more important in VR exploration involves projecting hand position along the normal vector of the surface. It arises when a normal vector is not parallel to the view vector of the user. When the user moves their hand horizontally, the movement of the adjusted

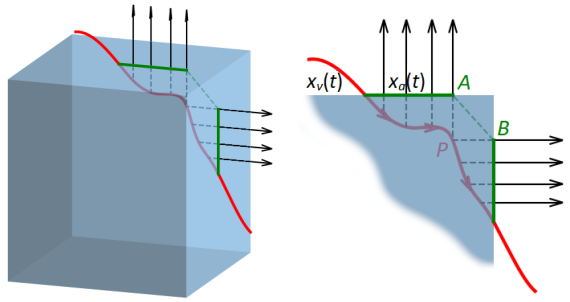


Fig. 2. Virtual hand trajectory $x_v(t)$ (red curve) and its projection $x_a(t)$ (green curve) onto the surfaces along their normal vectors (black arrows) in the case of the edge of two perpendicular surfaces. A line segment AB (green dots) shows projection discontinuity in such a case.

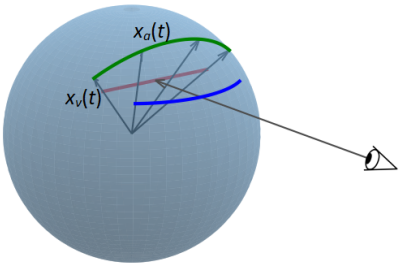


Fig. 3. Virtual hand trajectory $x_v(t)$ (red line), its projection $x_a(t)$ (green curve) on the surface along the surface normal vector, and the desired trajectory (blue curve) in the case where the normal vector is not parallel to the view vector of the user.

hand $x_a(t)$ is not horizontal when seen from the user's point of view, thus it seems unnatural and unclear to the user (Figure 3). To keep the expected horizontal movement, the projection onto the surface should be implemented along the view vector. This problem becomes significantly visible when the position of the virtual hand is deeply submerged in the object.

Another technique for stopping the hand from penetrating the object's surface is the *constraint-based* method. In this approach, the model of the character's hand consists of two hand meshes connected by a physics constraint. One mesh, the representation of the virtual hand $x_v(t)$, is the invisible target and controls the movement of the visible rendered hand $x_a(t)$ that is physically simulated to remain on the scene geometry and is limited by the constraint between the two meshes. The constraint is a linear or angular motor that tries to align the adjusted hand position $x_a(t)$ to the virtual hand position $x_v(t)$ in every frame. This method takes into account the previous configuration of the hand, and thus, the movement of the hand seems to be continuous. However, this technique can lead to the virtual hand jamming at a local minimum, which is problematic in VR applications and must be solved using other methods.

As both described approaches to hand position adjustment do not take into account either the importance of the continuous motion of the hand or the designer's intentions for the hand movement on the object, our goal was to develop a better method for determining the final hand position $x_a(t)$ in the scene to overcome the limitations mentioned above.

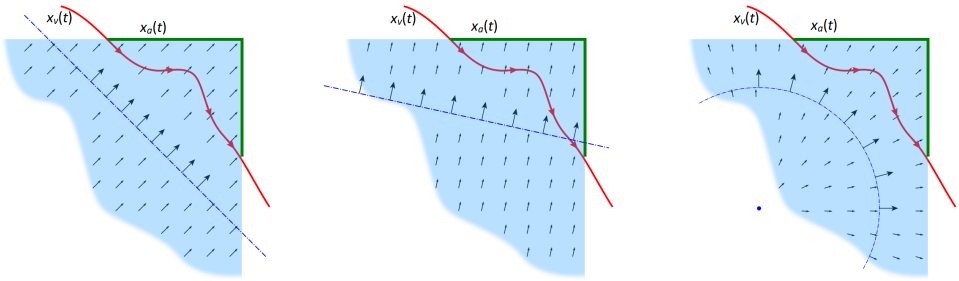


Fig. 4. Virtual hand trajectory $x_v(t)$ (red curve) and its projection $x_a(t)$ (green curve) on the surface using a vector field defined by projection shapes (in blue): a plane (left and center) and a sphere or a cylinder (right).

3.2 Algorithm description

The main contribution of our algorithm is to allow users to move their hands continuously over the surface of an object without visually penetrating its volume. Continuous motion is defined here as a curve without any breaks or jumps due to the application update time step. Our interaction technique feels more natural to the user and, in addition, can be represented visually, giving the scene designer an easy and convenient way to customize the desired user movement.

For simplicity, we model the virtual hand as a sphere, reducing its configuration space from position plus rotation to just the position component. Without loss of generality, we use this model to simplify the process of producing a smooth motion over arbitrary 3D objects. A complex representation of a virtual hand could be created using individual spheres connected in a tree hierarchy [Wang et al. 2017]. Since our algorithm utilizes the position of the user's hands, it is easily extended to use the deduced position of any objects held in the user's hands, such as a pen tip or probe.

We propose an approach in which, by introducing an auxiliary object that is the visual representation of projection, called a *projection shape*, we are able to provide a way of leading the user's hand in order to obtain a more natural trajectory. The idea itself is similar to the vector-field based approach [Morgenbesser and Srinivasan 1996]. A projection shape is a simplified version of a 3D object, one which may have better smoothness at vertices and along the edges than the triangulated mesh of an original object. Such a shape may look entirely different from the original one to force the desired behavior of the hand trajectory. Projection shapes can be defined in advance or precomputed using object geometry as input. In this work, we focus on projection methods and their correctness for the user's perception.

For each object in the scene, a projection shape is placed in its vicinity and used to project the trajectory back onto any intersected objects. Figure 4 shows simple projection shapes represented by primitives - a plane and a sphere or a cylinder placed in the corner of the object. Such a projection shape defines the vector field inside the object. The projected point x_a is then calculated by raycasting the hand's virtual position x_v onto the touched surface along the vector assigned to point x_v. The vector field equals 0 at points outside the object. In every time frame t, we perform the same calculations, obtaining a curve that represents the user's trajectory on the surface. For proposed projection shapes and defined regular vector fields, the trajectory appears continuous and smooth for the user, which guarantees a predicable movement speed in the scene without any visible jumps between adjacent positions.

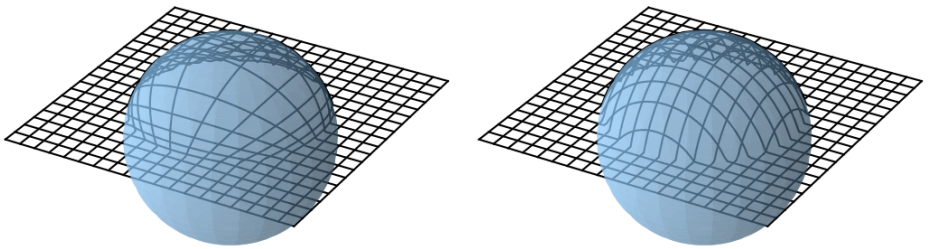


Fig. 5. A set of parallel lines projected on a sphere using a virtual sphere (left) and a virtual plane (right) as projection shapes.

Using the projection shapes, we can control and facilitate the user's motion to improve the user experience in VR. Motion over seemingly complex objects can be easily projected with primitive projection shapes. Objects of similar importance can be grouped by associating them with a common projection shape.

Moreover, a designer can change the properties of projection shapes (e.g. the plane's normal vector or the center of the sphere) to direct user's attention to a certain area of interest. After rotating the projection shape, from the left to center projection plane, as shown in Figure 4 - a different projected trajectory $x_a(t)$ is obtained. After rotation, the same hand motion $x_v(t)$ is projected primarily onto the top surface. Thus, by changing the normal of the projection plane, the hand motion can be attracted more by one of the surfaces, and the angles of the projection plane define the projection weights for given surfaces. It is worth mentioning that a potentially parallel projection plane to the top surface would render the right surface untouchable by the user. Such a procedure can be used to the designer's advantage to easily indicate which surfaces should be disabled for user interaction.

The projection shapes can also be used to parameterize the required movement on the touched surfaces. For instance, planes are strongly recommended when the designer aims to make it easier to draw parallel lines on curvilinear objects (Figure 5). They are also useful when it is necessary to keep the relation between the component parts of the hand as unchanged as possible.

Additionally, the designer might wish to employ the projection shapes to define a non-zero vector field in the vicinity of the object's surface. This will cause the hand to stick to the surface over a particular small area.

4 Evaluation

4.1 Testing process

Our algorithm was implemented in C++ and Unreal Engine 5, and evaluated on the basis of the test scenes, shown in Figure 6, to determine its behavior and effectiveness compared to the constraint-based and collision-less methods.

In each scene, we defined the *trajectories* that a user had to follow with their hand to investigate the nuances in the hand movement in different sliding interaction cases:

(1) The sphere scene includes a trajectory made up of five sphere parallels. This allows for the observation of differences in hand movement over the surface where the normal vector

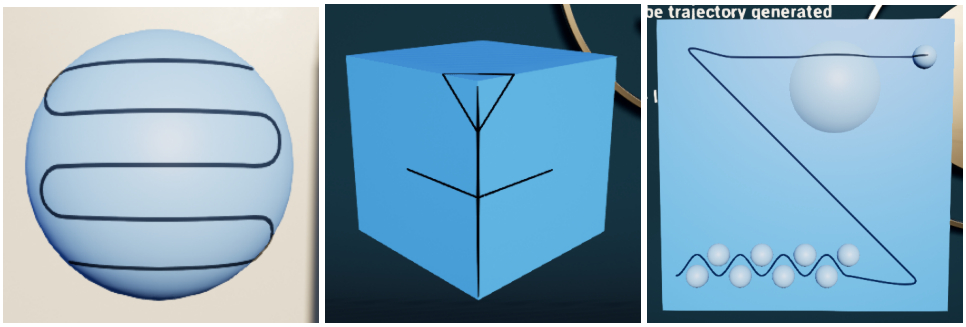


Fig. 6. Test scenes: the sphere (left), the box (middle), and the plane (right) with the reference trajectories (dark blue)

changes a lot, especially around the sphere's poles - the top and bottom parallels of the trajectory.

(2) The box scene focuses on areas with discontinuities in the normal vector. Three trajectories are defined: a horizontal one that intersects the edge, a vertical one that runs along the edge, and one that surrounds the corner of the cube.

(3) The plane scene emulates a case where the user is operating a panel with varying types of buttons on it. It allows us to see what differences there are between the various methods of resolving collisions when moving the hand over large areas where various bumps and obstacles are present. This test also allows the user's behavior and feelings to be better explored due to the longer time required to complete the test.

The task of the user was to reproduce the reference trajectory in such a way that the sphere representing the hand had to be tangent to the reference trajectory. In each scene, we tested various methods to support the user interactions: projection shapes as (1) a projection plane (PP), (2) a projection cylinder (PC) and (3) a projection sphere (PS), and compared them to state-of-the-art methods: (4) a constraint-based method (CB) and (5) a no-collision checking case (NC). These methods are hereafter referred to as *support methods*.

We also investigated *motion scenarios* to test static and dynamic interactions. The static scenario includes moving the hand along a reference trajectory without a time limit, while the dynamic one involves the user following an object (a small semi-transparent ball) moving at a constant speed along the trajectory. In the dynamic interaction, the users operated under pressure - they had to maintain the required speed of movement and try to keep to the imposed time limit. In the static case, any positional error was measured as the distance of the center of the adjusted hand to the nearest point on the reference trajectory. In the dynamic scenario, the error was calculated as the distance between the center of the hand and the center of the moving reference ball lying on the trajectory. The expected error was equal to the radius of the ball representing the hand.

Both scenarios were evaluated with the trajectory *preview mode* turned on and off. With the trajectory-off mode, the reference trajectory (in the static scenario) or the played movement (in the dynamic scenario) had to be memorized by the user after it having been displayed for a given period of time (10 seconds in the static case and for the duration of the animation in the dynamic one), and then it was reconstructed (imitated) by the user.

To sum up, each test configuration (called a *test* hereafter) consists of the scene with the trajectory, the support method, and the motion scenario. Combining all the possibilities - 5 trajectories, 5

support methods, two motion scenarios and two preview modes - produced 100 different test configurations.

The VR experiments were conducted on a group of 14 participants ranging from 20 to 46 years old with diverse experience in using VR and AR technologies and applications. Four participants described themselves as experts, while three rated their experience as above average, four as average or below average, and for three of them it was the first encounter with VR.

Before each test was performed, it was described in detail to the participant, but they had no opportunity to check the virtual environment before performing the task. Each test was repeated by users between 3 to 8 times, so it took more than two hours per person to carry out all test configurations. The results were then cleaned of anomalies - erroneous tests containing large motion noise caused e.g. by the user becoming distracted, or hardware errors, and then aggregated for individuals taking into account the motion scenario in a given preview mode, the trajectory on the scene, and the support method.

4.2 Quality results and discussion

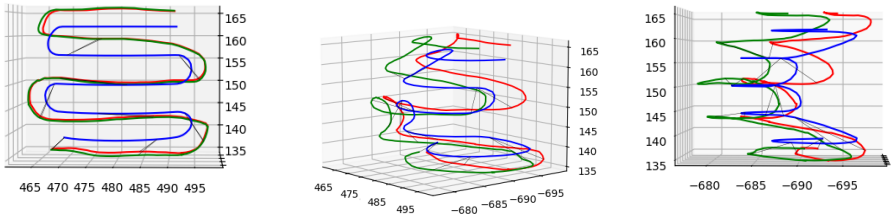
Figure 7 shows different views of the example trajectories recorded during one of the tests performed. The user hand was defined here as a sphere with a radius of 4, and trajectory $x_v(t)$ of the center of the virtual hand that was assumed to be tangent to the reference trajectory, was not constrained in any way during the test, contrary to the trajectory of the center $x_a(t)$ of the adjusted hand staying tangent to the surfaces in the scene.

The tests carried out proved that, due to the lack of physical constraints in the scene, users mostly pushed their hand into objects, and therefore the trajectory of the virtual hand (red) is inside the object, even behind the reference trajectory (blue). The adjusted trajectory (green) of the center of the virtual hand replicates the user's movement, as can be seen in the front view of the scene (left-hand figures), but is at the same time away by at least the distance of the radius of the hand (a value of 4) from the reference trajectory, not violating the constraints set by the object surfaces and satisfying the condition of sliding over them (right-hand figures).

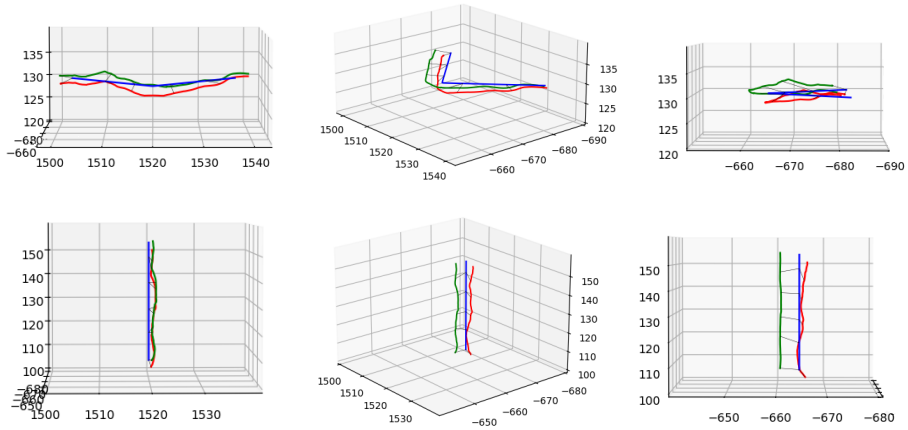
Figure 8 illustrates the distribution of distances from the center of the adjusted hand to the nearest point on the reference trajectory (Figures 8a-b) or to the center of the moving reference ball lying on the trajectory (Figures 8c-d). The median values are provided for each support method. The expected distance was equal to 4.

The non-collision method proved to be problematic as users had problems with stopping the hand on the surface of the objects due to the lack of any feedback. The median distances for the static scenario are equal to 2.8 (trajectory preview on) and 2.9 (preview off) proving that the hand was immersed in the object for most of its movement. For the dynamic case, the distances seem to be better (3.7 and 6.2, respectively), i.e. closer to or greater than the value of 4. However, it should be noted that in the dynamic case, the distances are measured to the reference ball, not to the nearest point on the surface, so higher values indicate a greater difference between these positions and do not guarantee the hand does not become immersed in the object. Without supporting the movement of the hand the users had a problem maintaining the correct distance to a target object or position - here the biggest difference in minimum and maximum distance was achieved in each scenario.

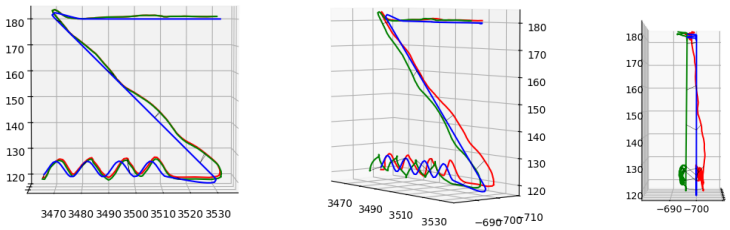
In contrast to the non-collision method, we can observe that all the projection shapes fulfilled their role in keeping the adjusted hand sliding on the surface of the objects. The median distances are 4.1 to 4.4 in the static case and 4.4 to 4.7 in the dynamic case with preview on, respectively, while the state-of-the-art constraint-based method achieved median distances of 4.2 and 4.4 in the static case and 5.0 in the dynamic with preview on.



(a) the sphere scene

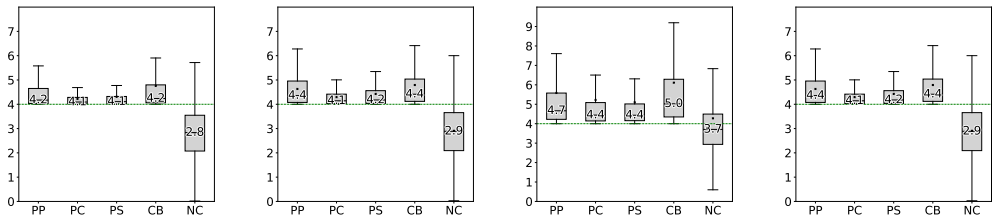


(b) the box scene



(c) the plane scene

Fig. 7. Examples of trajectories recorded for different scenes: a reference trajectory (blue), the center of the virtual hand $x_v(t)$ (red) and the center of the adjusted hand $x_a(t)$ (green).



(a) static, preview on (b) static, preview off (c) dynamic, preview on (d) dynamic, preview off

Fig. 8. Distances from the center of the adjusted hand to the point on the reference trajectory for different motion scenarios and different support methods: a projection plane (PP), a projection cylinder (PC), a projection sphere (PS), a constraint-based method (CB) and no-collision checking (NC).

The case studies proved that the constraint-based method gave even better results for particular parts of the trajectories than the projection shapes; however, it was not stable, especially in dynamic scenarios. Sometimes the adjusted hand would get stuck at a local minimum for a moment, spoiling immersion and the user experience. This is presented in Figure 9 in the lower part of the graphs for the part of the trajectory defined between the small buttons. The movement of the virtual hand (red) around the diagonal is very chaotic due to the attempts to free the adjusted hand (green) from between the buttons and move it along the diagonal. In the case of a discontinuous normal vector (e.g. the box scene), occasionally the adjusted hand penetrated scene objects or teleported between different positions. In the case of very fast actions, the movement of the adjusted hand sometimes lagged behind that of the virtual one; moreover, when the user changed the direction of movement, the virtual hand did not always move in the right direction because it experienced some inertia. This is the main reason why the distances for the dynamic scenario with preview enabled are larger for the constraint-based method than for the projection shapes.

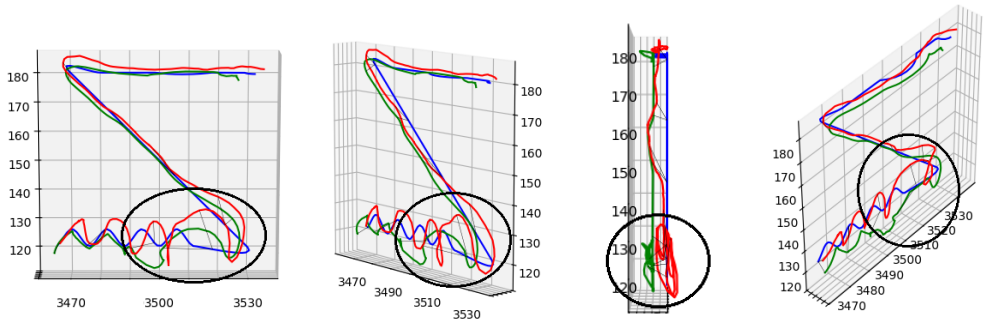


Fig. 9. Example of getting stuck in the constraint-based method: a reference trajectory (blue), the center of the virtual hand $x_v(t)$ (red) and the center of the adjusted hand $x_a(t)$ (green).

For the dynamic scenario with preview disabled, all the methods performed significantly worse. The median distances vary from 6.2 to 8.0 while the maximum values even exceeded 25.0. In this case, the results essentially report the ability of the participants to recall the movement of the reference ball (its trajectory and speed) instead of the properties of the investigated methods. Therefore, the results obtained for all methods are similar.

We focused not only on the accuracy obtained, but also on the users' experience. We observed that the results in the first seconds after the change in the motion scenario were worse for each person than in the later part. We attribute this to the time it takes users to adapt to the new interface. Even inexperienced participants quickly got used to the interfaces used in the application.

During an interview conducted after the experiment the participants repeatedly emphasized that it was easier to guide the hand over larger flat areas (like in the plane scene), compared to the scenes in which the slope of the object surface changed (the sphere scene) or the reference trajectory forced the need to make sudden changes in the direction of movement (the box scene). This is also confirmed by the slight variations in the hand trajectory deviation, noticeable in Figure 7c on the right when performing a slalom between buttons on the lower part of the plane. These were due, among other things, to the course of the corrected collision-free trajectory for the adjusted hand, which, due to the size of the hand sphere, which did not fit between the sphere-obstacles, could not run all the way tangent to the reference trajectory, and to the inability of the virtual hand to maintain a constant depth relative to the object plane.

4.3 Performance

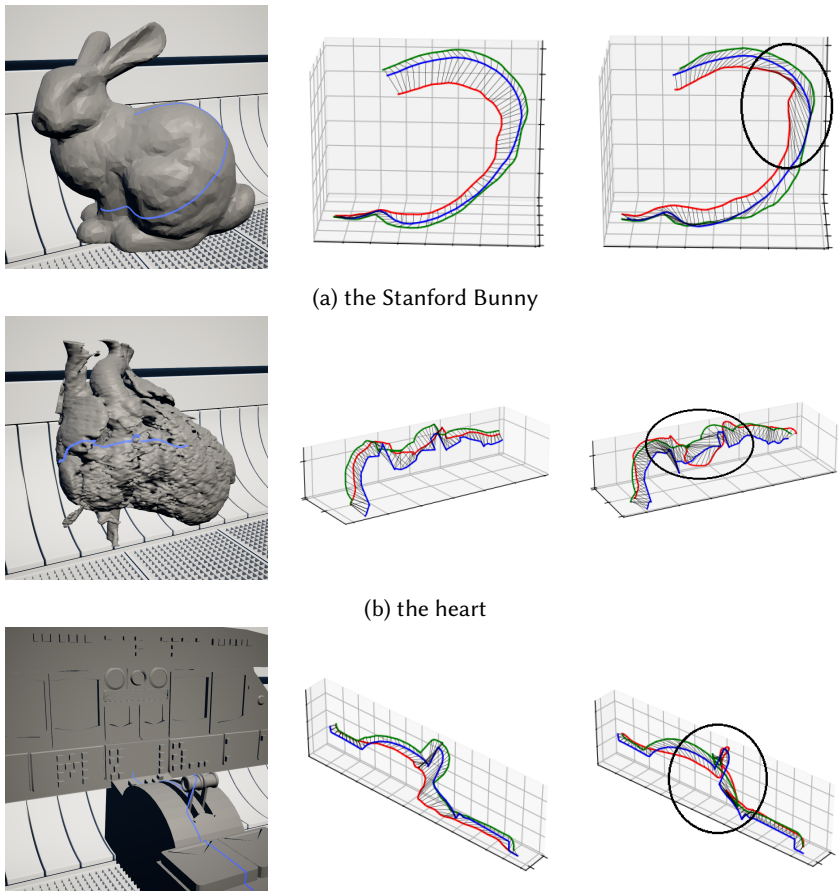
In order to measure how our method performs, we conducted experiments for three complex models. One, the Stanford Bunny, contains several indentations and protruding elements (particularly around the ears and feet), which may pose significant problems with hand wedging in traditional tracking solutions (Figure 10a). The anatomical heart model exhibits highly irregular triangulation and rough, uneven surfaces characteristic of automated segmentation processes, creating particularly challenging conditions for traditional constraint-based methods that often produce jittery hand movements when traversing these abrupt geometry changes (Figure 10b). The cockpit model represents a multi-object environment with varying topological elements ranging from planar surfaces (instrument panels) to cylindrical controls (throttle levers) and spherical components (control yokes) (Figure 10c).

For each model, we created three different levels of detail by adjusting triangle counts, which allowed us to analyze how our method scales with increasing geometric complexity. Then, for each level of detail and the dynamic motion scenario, we benchmarked projection shapes, the constraint-based and non-collision methods. The tests showed that our method works well with all complex models.

We evaluated the computational efficiency and real-time viability of our method via a series of benchmarks, contrasting its performance against the traditional constraint-based and non-collision approaches. The main objective of this work was to test the quality of the VR interaction method based on projection shapes; therefore, the implementation itself did not undergo advanced optimization. However, the frame rates presented in Table 1 showed that the performance of our method is on a par with the commonly used techniques. Across all tested configurations, the results indicate that the projection shapes maintain computational parity with constraint-based techniques, exhibiting no measurable increase in overhead. Even in the presence of complex geometries and high-detail meshes, performance remains stable. Tests performed on Oculus Quest 2 with a 72 Hz frame rate cap showed FPS consistently oscillating around 72, meaning the frame rate did not drop below what was required for a smooth VR experience.

5 Conclusion and Future Work

We designed and developed a new VR interaction method based on projection shapes to correct the user's hand position when interacting with the surface of virtual scene objects. The auxiliary projection shapes define the vector field used to calculate a corrected position of the hand. The developed method was tested with 14 participants to compare the projection shapes with the most



(a) the Stanford Bunny

(b) the heart

(c) the cockpit

Fig. 10. Test models (left) and examples of trajectories: a reference trajectory (blue), the center of the virtual hand $x_v(t)$ (red) and the center of the adjusted hand $x_a(t)$ (green), recorded during a dynamic scenario for projection shapes (middle) and constraint-based (right) methods. Problems with hand wedging in the constraint-based method are highlighted.

commonly used constraint-based correction method. The experiments show that our method gives results similar to the constraint-based one. The projection shapes aided the users in keeping the hand on the surface of the objects while moving. In addition, they provided the most stable results regardless of the test case, increasing realism and improving the user experience.

In virtual reality, especially in a dynamic environment, the user may not have enough time to interact with objects precisely. The dynamic scenario tests, designed as a simulation of such situations, required, on the one hand, precision of movement and interaction, and, on the other hand, quick execution of a given task. The experiments conducted in the dynamic scenario proved that the projection shapes guarantee smooth movement of the adjusted hand, without noticeable problems getting stuck or delays.

Furthermore, the user's hand motion is usually imperfect and not very accurate, especially over small objects such as buttons in a virtual plane's cockpit. Smooth motion of the virtual hand

Table 1. Frame rates (FPS) for the dynamic motion scenario and different support methods for various mesh densities; $\#V$ - the number of vertices, $\#T$ - the number of triangles. All tests were performed on an Intel Core i7-10870H at 2.2 GHz, 32 GB RAM, and an NVIDIA GeForce RTX 3070 Laptop graphics card with 8 GB, running within Unreal Engine 5.0. The Oculus Quest 2 served as the target head-mounted display.

		Projection shapes			Constraint-based			Non-collision		
$\#V$	$\#T$	Avg FPS	+90	+60	Avg FPS	+90	+60	Avg FPS	+90	+60
the Stanford Bunny										
14K	5K	72.2	0.14%	100%	72.3	0.30%	100%	72.1	0.14%	100%
29K	41K	72.2	0.14%	100%	72.2	0.14%	100%	72.3	1.10%	99%
259K	405K	72.3	0.30%	100%	72.3	0.14%	100%	72.1	0.14%	100%
the heart										
2K	5K	72.0	0.14%	100%	71.9	0.14%	100%	72.1	0.14%	100%
20K	41K	72.1	0.43%	100%	72.0	0.0%	100%	72.0	0.14%	100%
184K	369K	72.1	0.28%	100%	72.1	0.14%	100%	72.0	0.14%	100%
the cockpit										
11K	5K	72.0	0.14%	100%	72.0	0.28%	100%	72.0	0.14%	100%
43K	43K	72.0	0.14%	100%	72.1	0.14%	100%	72.1	0.28%	100%
273K	414K	71.9	0.42%	99%	71.3	0.57%	98%	72.0	0.70%	100%

supported by the projection shapes can improve the physical-based selection mechanism. The user can simply slide through the interface, while the hand position is automatically adjusted to the points of interest. This technique can then improve the selection process and, moreover, can also be used directly to calculate force feedback, which can be exploited by haptic devices.

The results show that our approach improves the quality of interactions with complex models while having no significant impact on performance. Our method generalizes and scales well even for complex models, making it a great solution for applications ranging from medical visualization to industrial training simulations.

Our method is useful for interface designers who are responsible for creating VR experiences. They will be able to design the required user's hand motion over objects as a pre-processing step in a very convenient visual manner during scene creation. The trajectories of hands are controlled by choosing different vector fields defined by projection shapes. The change in projection shape modifies the behavior of the user's hand - its trajectory will be different even for visually identical scenes. In this way, designers can easily influence the hand movement and aid e.g. writing or drawing with a pen tip on curvilinear surfaces by making the surface virtually more flat by employing projection shapes with smaller curvatures than the object the user interacts with.

Although designers are able to visually influence the way the user's hand runs over surfaces, there are, however, potential improvements that we would like to mention. One open question is how to automatically select the best projection shape for each object or a group of objects because the accuracy achieved varies among the pairs *projection shape - scene object* they are used for. In the case of projection shapes that are not adapted to the scene objects, small differences in hand movement can drastically change the projected trajectory. Moreover, more complex projection primitives (e.g. capsules) could also be investigated. The next step after automatic calculation of the projection shapes could be expanding the system for dynamically changing the properties of the projection shapes during the run-time depending e.g. on the user's view direction.

References

Ferran Argelaguet and Carlos Andujar. 2013. A survey of 3D object selection techniques for virtual environments. *Computers & Graphics* 37 (5 2013), 121–136. Issue 3. doi:10.1016/j.cag.2012.12.003

Ferran Argelaguet, Ludovic Hoyet, Michael Trico, and Anatole Lecuyer. 2016. The role of interaction in virtual embodiment: Effects of the virtual hand representation. In *2016 IEEE Virtual Reality (VR)*. 3–10. doi:10.1109/VR.2016.7504682

David Baraff. 1993. Non-penetrating Rigid Body Simulation. In *EUROGRAPHICS'93: State of the Art Reports. Barcelona, September 6-10, 1993 (1993.Barcelona)*. [S.n.], Chapter 2.

David Baraff. 1996. Linear-time dynamics using Lagrange multipliers. In *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '96)*. Association for Computing Machinery, New York, NY, USA, 137–146. doi:10.1145/237170.237226

Lorenzo Bartalucci, Nicola Secciani, Chiara Brogi, Alberto Topini, Andrea Della Valle, Alessandro Ridolfi, and Benedetto Allotta. 2023. An original mechatronic design of a kinaesthetic hand exoskeleton for virtual reality-based applications. *Mechatronics* 90 (4 2023), 102947. doi:10.1016/j.mechatronics.2023.102947

Mourad Bouzit, Grigore Burdea, George Popescu, and Rares Boian. 2002. The Rutgers Master II-new design force-feedback glove. *IEEE/ASME Transactions on mechatronics* 7 (2002), 256–263. Issue 2.

Matthias Buck and Elmar Schömer. 1998. Interactive rigid body manipulation with obstacle contacts. *The Journal of Visualization and Computer Animation* 9, 4 (1998), 243–257. doi:10.1002/(SICI)1099-1778(1998100)9:4<243::AID-VIS189>3.0.CO;2-5

Manuel Caeiro-Rodríguez, Iván Otero-González, Fernando A. Mikic-Fonte, and Martín Llamas-Nistal. 2021. A Systematic Review of Commercial Smart Gloves: Current Status and Applications. *Sensors* 21 (4 2021), 2667. Issue 8. doi:10.3390/s21082667

Weiya Chen, Chenchen Yu, Chenyu Tu, Zehua Lyu, Jing Tang, Shiqi Ou, Yan Fu, and Zhidong Xue. 2020. A Survey on Hand Pose Estimation with Wearable Sensors and Computer-Vision-Based Methods. *Sensors* 20 (2 2020), 1074. Issue 4. doi:10.3390/s20041074

Inrak Choi, Elliot W. Hawkes, David L. Christensen, Christopher J. Ploch, and Sean Follmer. 2016. Wolverine: A wearable haptic interface for grasping in virtual reality. In *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 986–993. doi:10.1109/IROS.2016.7759169

Scott Devine, Karen Rafferty, and Stuart Ferguson. 2017. "HapticVive" - A point contact encounter haptic solution with the HTC VIVE and Baxter Robot. In *International Conference in Central Europe on Computer Graphics, Visualization and Computer Vision WSCG 2017: Proceedings* (may ed.), Vaclav Skala (Ed.), Vol. 2702. University of West Bohemia, 17–24.

Epic Games, Inc. 2023. *Collision overview in Unreal Engine*. Epic Games, Inc. https://dev.epicgames.com/documentation/en-us/unreal-engine/collision-in-unreal-engine---overview

Linus Franke, Laura Fink, Jana Martschinke, Kai Selgrad, and Marc Stamminger. 2021. Time-Warped Foveated Rendering for Virtual Reality Headsets. *Computer Graphics Forum* 40 (2 2021), 110–123. Issue 1. doi:10.1111/cgf.14176

Janet K. Gibbs, Marco Gillies, and Xueni Pan. 2022. A comparison of the effects of haptic and visual feedback on presence in virtual reality. *International Journal of Human Computer Studies* 157 (1 2022). doi:10.1016/j.ijhcs.2021.102717

Mikako Harada, Andrew Witkin, and David Baraff. 1995. Interactive physically-based manipulation of discrete/continuous models. In *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '95)*. Association for Computing Machinery, New York, NY, USA, 199–208. doi:10.1145/218380.218443

Markus Holl, Markus Oberweger, Clemens Arth, and Vincent Lepetit. 2018. Efficient Physics-Based Implementation for Realistic Hand-Object Interaction in Virtual Reality. In *2018 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. IEEE, 175–182. doi:10.1109/VR.2018.8448284

Matthias Hoppe, Pascal Knierim, Thomas Kosch, Markus Funk, Lauren Futami, Stefan Schneegass, Niels Henze, Albrecht Schmidt, and Tonja Machulla. 2018. VRHapticDrones: Providing Haptics in Virtual Reality through Quadcopters. In *Proceedings of the 17th International Conference on Mobile and Ubiquitous Multimedia* (Cairo, Egypt) *(MUM '18)*. Association for Computing Machinery, New York, NY, USA, 7–18. doi:10.1145/3282894.3282898

Zhiming Hu. 2020. Gaze Analysis and Prediction in Virtual Reality. In *2020 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*. 543–544. doi:10.1109/VRW50115.2020.00123

Robert Iacob, Diana Popescu, Frederic Noel, Thibault Louis, Cedric Masclet, and Patrick Maigrot. 2014. Haptic Devices Evaluation for Industrial Use. In *EuroVR 2014 - Conference and Exhibition of the European Association of Virtual and Augmented Reality*, Jerome Perret, Valter Basso, Francesco Ferrise, Kaj Helin, Vincent Lepetit, James Ritchie, Christoph Runde, Mascha van der Voort, and Gabriel Zachmann (Eds.). The Eurographics Association. doi:/10.2312/eurovr.20141337

Jan Jacobs and Bernd Froehlich. 2011. A soft hand model for physically-based manipulation of virtual objects. In *2011 IEEE Virtual Reality Conference*. IEEE, 11–18. doi:10.1109/VR.2011.5759430

Namkyoo Kang, Young June Sah, and Sangwon Lee. 2021. Effects of visual and auditory cues on haptic illusions for active and passive touches in mixed reality. *International Journal of Human Computer Studies* 150 (6 2021). doi:10.1016/j.ijhcs.2021.102613

Anuradha Kar and Peter Corcoran. 2017. A Review and Analysis of Eye-Gaze Estimation Systems, Algorithms and Performance Evaluation Methods in Consumer Platforms. *IEEE Access* 5 (2017), 16495–16519. doi:10.1109/ACCESS.2017.2735633

Jun Sik Kim and Jung Min Park. 2015. Physics-based hand interaction with virtual objects. In *Proceedings - IEEE International Conference on Robotics and Automation*, Vol. 2015-June. Institute of Electrical and Electronics Engineers Inc., 3814–3819. Issue June. doi:10.1109/ICRA.2015.7139730

Yongwan Kim and Jinah Park. 2014. Study on interaction-induced symptoms with respect to virtual grasping and manipulation. *International Journal of Human-Computer Studies* 72 (2 2014), 141–153. Issue 2. doi:10.1016/j.ijhcs.2013.09.011

Hugh B. Morgenbesser and Mandayam A. Srinivasan. 1996. Force Shading for Haptic Shape Perception. In *ASME International Mechanical Engineering Congress and Exposition*, Vol. Dynamic Systems and Control. 407–412. doi:10.1115/IMECE1996-0363

Jun Murayama, Laroussi Bougrila, Katsuhito Akahane, Shoichi Hasegawa, and Makoto Sato. 2004. SPIDAR G&G: A Two-Handed Haptic Interface for Bimanual VR Interaction. In *Proceedings of Euro-Haptics*.

Oliver Ozioko and Ravinder Dahiya. 2022. Smart Tactile Gloves for Haptic Interaction, Communication, and Rehabilitation. *Advanced Intelligent Systems* 4 (2 2022), 2100091. Issue 2. doi:10.1002/aisy.202100091

Claudio Pacchierotti, Stephen Sinclair, Massimiliano Solazzi, Antonio Frisoli, Vincent Hayward, and Domenico Prattichizzo. 2017. Wearable Haptic Systems for the Fingertip and the Hand: Taxonomy, Review, and Perspectives. *IEEE Transactions on Haptics* 10 (10 2017), 580–600. Issue 4. doi:10.1109/TOH.2017.2689006

Mingzhang Pan, Yingzhe Tang, and Hongqi Li. 2022. Review of the State-of-the-Art of Data Gloves. In *2022 IEEE International Symposium on Smart Electronic Systems (iSES)*. 402–405. doi:10.1109/iSES54909.2022.00088

J. Perret and E. Vander Poorten. 2018. Touching Virtual Reality: A Review of Haptic Gloves. In *ACTUATOR 2018; 16th International Conference on New Actuators*. 1–5.

Mores Prachyabrued and Christoph W. Borst. 2012. Virtual grasp release method and evaluation. *International Journal of Human Computer Studies* 70 (11 2012), 828–848. Issue 11. doi:10.1016/j.ijhcs.2012.06.002

Andreas Pusch, Olivier Martin, and Sabine Coquillart. 2009. HEMP-hand-displacement-based pseudo-haptics: A study of a force field application and a behavioural analysis. *International Journal of Human Computer Studies* 67 (3 2009), 256–268. Issue 3. doi:10.1016/j.ijhcs.2008.09.015

Jiaming Qi, Feng Gao, Guanghui Sun, Joo Chuan Yeo, and Chwee Teck Lim. 2023. HaptGlove—Untethered Pneumatic Glove for Multimode Haptic Feedback in Reality–Virtuality Continuum. *Advanced Science* 10 (9 2023). Issue 25. doi:10.1002/advs.202301044

Karen Rafferty, Scott Devine, and Daniel Brice. 2017. A Novel Force Feedback Haptics System with Applications in Phobia Treatment. In *WSCG 2017*.

Stéphane Redon, Abderrahmane Kheddar, and Sabine Coquillart. 2002. Fast Continuous Collision Detection between Rigid Bodies. *Computer Graphics Forum* 21, 3 (2002), 279–287. doi:10.1111/1467-8659.t01-1-00587

Donghyun Sim, Yoonchul Baek, Minjeong Cho, Sunghoon Park, A. S. M. Sharifuzzaman Sagar, and Hyung Seok Kim. 2021. Low-Latency Haptic Open Glove for Immersive Virtual Reality Interaction. *Sensors* 21, 11 (2021). doi:10.3390/s21113682

Mandayam A. Srinivasan and Cagatay Basdogan. 1997. Haptics in virtual environments: Taxonomy, research status, and challenges. *Computers & Graphics* 21, 4 (1997), 393–404. doi:10.1016/S0097-8493(97)00030-7 Haptic Displays in Virtual Environments and Computer Graphics in Korea.

Yudai Tanaka, Alan Shen, Andy Kong, and Pedro Lopes. 2023. Full-Hand Electro-Tactile Feedback without Obstructing Palmar Side of Hand. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 80, 15 pages. doi:10.1145/3544548.3581382

Qianqian Tong, Wenxuan Wei, Yuru Zhang, Jing Xiao, and Dangxiao Wang. 2023. Survey on Hand-Based Haptic Interaction for Virtual Reality. *IEEE Transactions on Haptics* 16, 2 (2023), 154–170. doi:10.1109/TOH.2023.3266199

Russell Turner and Enrico Gobbetti. 1998. Interactive Construction and Animation of Layered Elastically Deformable Characters. 135-152 pages. Issue 2.

Unity Technologies 2023. *Unity Manual: Collision*. Unity Technologies. https://docs.unity3d.com/Manual/collision-section.html

J. M. P. van Waveren. 2016. The asynchronous time warp for virtual reality on consumer hardware. In *Proceedings of the 22nd ACM Conference on Virtual Reality Software and Technology* (Munich, Germany) *(VRST '16)*. Association for Computing Machinery, New York, NY, USA, 37–46. doi:10.1145/2993369.2993375

Mickeal Verschaar, Daniel Lobo, and Miguel A. Otaduy. 2018. Soft Hand Simulation for Smooth and Robust Natural Interaction. In *25th IEEE Conference on Virtual Reality and 3D User Interfaces, VR 2018 - Proceedings*. Institute of Electrical and Electronics Engineers Inc., 183–190. doi:10.1109/VR.2018.8447555

Ethan Waisberg, Joshua Ong, Nasif Zaman, Sharif Amit Kamran, Prithul Sarker, Alireza Tavakkoli, and Andrew G. Lee. 2024. Head-mounted display cataract surgery: a new frontier with eye tracking and foveated rendering technology. *Eye (Basingstoke)* 38, 5 (April 2024), 1022–1023. doi:10.1038/s41433-023-02794-4

Dangxiao Wang, Meng Song, Afzal Naqash, Yukai Zheng, Weiliang Xu, and Yuru Zhang. 2019. Toward Whole-Hand Kinesthetic Feedback: A Survey of Force Feedback Gloves. *IEEE Transactions on Haptics* 12 (4 2019), 189–204. Issue 2. doi:10.1109/TOH.2018.2879812

Dangxiao Wang, Xiaohan Zhao, Youjiao Shi, Yuru Zhang, and Jing Xiao. 2017. Six Degree-of-Freedom Haptic Simulation of a Stringed Musical Instrument for Triggering Sounds. *IEEE Transactions on Haptics* 10, 2 (2017), 265–275. doi:10.1109/TOH.2016.2628369

Lili Wang, Xuehuai Shi, and Yi Liu. 2023. Foveated rendering: A state-of-the-art survey. *Computational Visual Media* 9 (6 2023), 195–228. Issue 2. doi:10.1007/s41095-022-0306-4

Yu Wang, Ziran Hu, Shouwen Yao, and Hui Liu. 2022. Using visual feedback to improve hand movement accuracy in confined-occluded spaces in virtual reality. *Visual Computer* (2022). doi:10.1007/s00371-022-02424-2

Minglu Zhu, Zhongda Sun, Zixuan Zhang, Qiongfeng Shi, Tianyiyi He, Huicong Liu, Tao Chen, and Chengkuo Lee. 2020. Haptic-feedback smart glove as a creative human-machine interface (HMI) for virtual/augmented reality applications. *Science Advances* 6, 19 (2020), eaaz8693. doi:10.1126/sciadv.aaz8693